ShrinkageTrees: An R Package for Bayesian Tree Ensembles for Survival Analysis and Causal Inference

by Tijn Jacobs, Stéphanie van der Pas, and Wessel van Wieringen

Abstract ShrinkageTrees is an R package for fitting Bayesian tree ensemble models with flexible regularisation mechanisms for survival analysis and causal inference. The package provides unified implementations of BART, DART, and Bayesian Causal Forests (BCF) for right-censored and interval-censored survival outcomes via accelerated failure time (AFT) formulations. Multiple regularisation strategies can be combined within a single model: structural sparsity through tree depth priors and Dirichlet splitting priors (DART), and parametric shrinkage on the leaf step heights through global–local scale mixture priors. A key contribution is the implementation of Horseshoe Forests, which place a horseshoe prior directly on the step heights, enabling adaptive shrinkage that aggressively regularises uninformative leaves while preserving strong signals — a strategy particularly suited to high-dimensional settings. An efficient Rcpp backend and coherent S3 interface support reproducible workflows for both prediction and causal effect estimation.

1 Introduction

Bayesian Additive Regression Trees [BART; Chipman et al. (2010)] model the response as a sum of m shallow decision trees, each regularised through a Bayesian prior. BART captures complex interactions and non-linearities without specifying a parametric form. This makes BART well suited to settings where the relationship between covariates and the response is unknown, including survival analysis. The **BART** package (Sparapani et al., 2021) is, to our knowledge, the only CRAN package that implements Bayesian tree ensembles for survival data. It supports discrete-time hazard and AFT formulations, competing risks, and recurrent events. However, it does not support interval-censored outcomes, causal inference models, or global–local shrinkage priors on the leaf parameters. We address all three limitations in **ShrinkageTrees**.

BART has become a popular tool for causal inference (Hill, 2011; Dorie et al., 2019). Hahn et al. (2020) introduce Bayesian Causal Forests (BCF), which decompose the response surface into a prognostic function and a treatment effect function. This separation improves estimation of heterogeneous treatment effects. The **bcf** and **stochtree** packages implement BCF for continuous outcomes. Causal inference and survival analysis arise together naturally in clinical trials and observational studies, where the outcome is a time-to-event and treatment effects may vary across patient subgroups. No existing R package implements Bayesian causal forests for survival endpoints.

High-dimensional settings, where the number of predictors p is large relative to the sample size n, demand adequate regularisation. We distinguish two components on which regularisation can act in Bayesian tree ensembles. The first is the tree structure, which determines how the covariate space is partitioned. The second is the leaf parameters, which determine each partition's contribution to the fitted function. Standard BART regularises both components generically: a depth prior discourages complex trees, and a conjugate normal prior shrinks the leaf parameters. DART (Linero, 2018) strengthens structural regularisation by placing a Dirichlet prior on the splitting probabilities, encouraging the model to split on a smaller subset of relevant predictors. Jacobs et al. (2025) target the second component and place global–local shrinkage priors, such as the horseshoe (Carvalho et al., 2010), directly on the leaf parameters. This shrinks weak or spurious contributions aggressively toward zero while preserving strong signals. We refer to tree ensembles with such priors as *Horseshoe Forests*.

We introduce **ShrinkageTrees**, a unified R package that addresses these three gaps. We support survival outcomes via AFT models with both right-censoring and interval-censoring. We extend BCF to survival endpoints, enabling estimation of heterogeneous treatment effects for time-to-event outcomes. We decouple structural sparsity from parametric shrinkage and implement Horseshoe Forests, allowing both forms of regularisation to be combined within a single model. We implement the core samplers in C++ via **Rcpp** (Eddelbuettel and François, 2011) and provide a coherent S3 interface with `print`, `summary`, `predict`, and `plot` methods. Jacobs et al. (2025) develop the methodological foundations.

We introduce **ShrinkageTrees**, a unified R package that addresses the three gaps identified above. We extend BART to survival outcomes via AFT models for both right-censored and interval-censored data. We extend the BCF decomposition to survival endpoints, providing, to our knowledge, the first causal forest implementation for time-to-event outcomes in R. We implement both Dirichlet

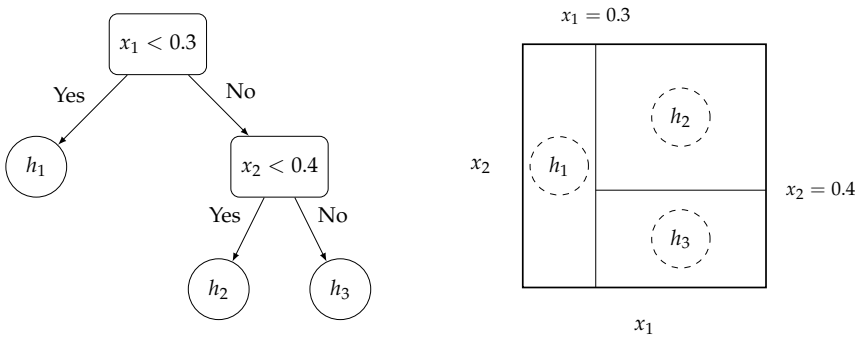

Figure 1: Schematic of a single regression tree. Interior nodes contain binary splitting rules of the form $x_\rho < c$; terminal nodes (leaves) contain step heights h_ℓ. An observation traverses the tree from root to leaf, and its prediction is the step height of the leaf it reaches.

splitting priors and Horseshoe Forests, allowing both forms of regularisation to be combined freely within a single model. The Horseshoe Forest sampler relies on an efficient reversible jump algorithm implemented in C++ via **Rcpp** (Eddelbuettel and François, 2011), and is designed to be extendable to other global–local shrinkage priors. We provide a coherent S3 interface with `print`, `summary`, `predict`, and `plot` methods.

The remainder of this paper is organised as follows. Section 2 provides a brief overview of the statistical methodology. Section 3 reviews related R packages. Section 4 describes the package architecture. Section 5 demonstrates the package on a high-dimensional ovarian cancer survival dataset. Section 6 concludes.

2 Overview of Statistical Methodology

Bayesian Additive Regression Trees

We adapt the Bayesian Additive Regression Trees (BART) model, originally introduced by Chipman et al. (2010) to estimate the conditional mean function $\mathbb{E}[Y_i \mid \mathbf{x}_i]$ in regression settings with continuous uncensored outcomes. The model assumes

$$Y_i = \sum_{j=1}^{m} g(\mathbf{x}_i;\, \mathcal{T}_j, \mathcal{H}_j) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2),$$

where each $\mathcal{T}_j$ is a binary decision tree that defines recursive partitioning rules over the covariate space, and $\mathcal{H}_j = \{h_{j1}, \ldots, h_{jL_j}\}$ denotes the set of step heights associated with the L_j leaves of tree $\mathcal{T}_j$.

For a given input $\mathbf{x}$, the function $g(\mathbf{x};\, \mathcal{T}_j, \mathcal{H}_j)$ maps $\mathbf{x}$ to a leaf node ℓ in $\mathcal{T}_j$ and returns the corresponding value $h_{j\ell}$. Each tree partitions the covariate space into disjoint regions, assigning a constant prediction to each (see Figure 1). The prediction of the j-th tree for individual i is obtained by

$$g(\mathbf{x}_i;\, \mathcal{T}_j, \mathcal{H}_j) = \sum_{\ell=1}^{L_j} h_{j\ell} \cdot \mathbb{1}(\mathbf{x}_i;\, \mathcal{T}_j, \ell),$$

where $\mathbb{1}(\mathbf{x}_i;\, \mathcal{T}_j, \ell) = 1$ if $\mathbf{x}_i$ is assigned to leaf ℓ in tree $\mathcal{T}_j$, and zero otherwise.

A prior is specified over the ensemble $\{(\mathcal{T}_j, \mathcal{H}_j)\}_{j=1}^{m}$ by independently assigning a prior on the tree structure and a conditional prior on the step heights given the tree:

$$p(\mathcal{T}_j, \mathcal{H}_j) = p_{\mathcal{T}}(\mathcal{T}_j)\, p_{\mathcal{H}}(\mathcal{H}_j \mid \mathcal{T}_j) \quad \text{for } j = 1, \ldots, m.$$

These define the two key components of the BART prior. The tree structure $\mathcal{T}$ of each tree is drawn from a prior $p_{\mathcal{T}}(\mathcal{T})$ originally described by Chipman et al. (1998), based on a heterogeneous Galton–Watson branching process (Ročková and Saha, 2020). The process begins at a root node and recursively decides whether to split a node based on its depth d. The probability that a node at depth d is non-terminal is given by

$$p(d) = \alpha\,(1+d)^{-\beta},$$

with hyperparameters $\alpha > 0$ and $\beta > 0$ governing the expected depth and shape of the tree. Given

a tree topology, each interior node is assigned a binary decision rule of the form $[x_\rho \leq c]$, where the splitting variable ρ is selected uniformly from the available covariates and the split point c is drawn uniformly from the range of observed values for variable ρ. The splitting rules are constrained to those producing valid partitions, that is, each subset contains a minimal chosen number of observations.

The prior $p_{\mathcal{H}}$ on the step heights $\mathcal{H}$ is defined conditionally on the tree structure $\mathcal{T}$. The step heights $h_{j\ell}$ are assigned independent Gaussian priors of the form

$$h_{j\ell} \sim \mathcal{N}\left(0, \frac{1}{4\sqrt{m}}\right),$$

where the variance is scaled to ensure that the prior places approximately 95% of its mass within the observed range of the response. This prior is conjugate, enabling efficient non-reversible updates using a Gibbs sampler.

The noise is modelled as $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$, with the error variance σ^2 assigned a scaled inverse-chi-squared prior:

$$\sigma^2 \sim \text{Inv-}\chi^2(\nu, \psi),$$

where ν denotes the degrees of freedom parameter and ψ the prior scale. We follow Chipman et al. (2010), setting $\nu = 3$ and choosing ψ so that the prior is centred around a rough estimate of σ^2.

Regularisation

The parameters of a BART ensemble fall into two groups: tree structure parameters (nodes, splitting variables, split points) that determine how the covariate space is partitioned, and step heights that determine each partition's contribution to the fitted function. Regularisation can target either group.

Structural regularisation. Standard BART regularises the tree structure through the branching process prior of Chipman et al. (2010): each internal node at depth d splits with probability $\alpha(1+d)^{-\beta}$, where $\alpha \in (0,1)$ and $\beta \geq 0$ discourage deep trees. Under the default values $\alpha = 0.95$ and $\beta = 2$, most trees remain shallow. DART (Linero, 2018) strengthens structural regularisation by replacing the uniform distribution over splitting variables with a Dirichlet prior on the splitting probabilities $\mathbf{s} = (s_1, \ldots, s_p)$. By concentrating the prior on sparse probability vectors, DART encourages the ensemble to split on a subset of relevant variables, achieving structural variable selection.

Shrinkage on step heights. Rather than regularising the tree structure alone, we can place shrinkage priors directly on the leaf step heights. **ShrinkageTrees** adopts a scale mixture of normals representation,

$$h_\ell \mid \lambda_\ell, \tau, \omega \sim \mathcal{N}(0, \omega\, \lambda_\ell^2\, \tau^2),$$

where τ is a global shrinkage parameter shared across all leaves, λ_ℓ is a local scale specific to leaf ℓ, and ω is a fixed scaling constant. The global parameter τ controls the overall degree of regularisation, while the local parameters λ_ℓ allow individual leaves with strong signals to escape shrinkage.

The horseshoe prior (Carvalho et al., 2010) arises as a specific instantiation of this framework by placing independent half-Cauchy priors on both the local and global scales:

$$\lambda_\ell \sim \mathrm{C}^+(0, \alpha_\lambda), \qquad \tau \sim \mathrm{C}^+(0, \alpha_\tau),$$

where $\mathrm{C}^+(0, \alpha)$ denotes the half-Cauchy distribution with scale α. The half-Cauchy has its median at α and possesses heavy tails — its mean and variance do not exist — which allows the local scales to take very large values when the data support a strong effect (Polson and Scott, 2012). This combination aggressively shrinks the contributions of uninformative leaves towards zero while preserving large, data-supported signals. Following Jacobs et al. (2025), we set $\alpha_\tau = \alpha_\lambda = k/\sqrt{m}$. The Dirichlet and horseshoe priors address complementary aspects of regularisation — structural sparsity and parametric shrinkage, respectively — and can be combined within a single model in **ShrinkageTrees**.

Prediction and causal models

The BART formulation above estimates a single function of the covariates — we refer to this as the *prediction model*. For causal inference, we adopt the Bayesian Causal Forest [BCF; Hahn et al. (2020)] decomposition, also known as the τ-learner (Caron et al., 2022). Given a binary treatment indicator $A_i \in \{0, 1\}$ and an estimate of the propensity score $\hat{e}(\mathbf{x}_i) = \hat{P}(A_i = 1 \mid \mathbf{x}_i)$, the *causal model* writes

$$y_i = \mu(\mathbf{x}_i, \hat{e}(\mathbf{x}_i)) + A_i\, \tau(\mathbf{x}_i) + \varepsilon_i,$$

where μ is a prognostic function modelled by a forest of m_μ trees and τ is a treatment effect function modelled by a separate forest of m_τ trees. This separation allows the prognostic and treatment effect

components to have different regularisation and complexity. The conditional average treatment effect is $\mathrm{CATE}(\mathbf{x}) = \tau(\mathbf{x})$, and the average treatment effect is $\mathrm{ATE} = \mathbb{E}[\tau(\mathbf{X})]$.

More generally, the scalar multiplier on τ in the BCF decomposition need not equal the raw treatment indicator. Writing b_i for the coding applied to subject i, the model becomes $y_i = \mu(\mathbf{x}_i, \hat{e}(\mathbf{x}_i)) + b_i\,\tau(\mathbf{x}_i) + \varepsilon_i$. Hahn et al. (2020) discuss several choices. *Centered* coding sets $b_i = A_i - 1/2$, which is the original BCF parameterisation and the default in **ShrinkageTrees**. *Binary* coding uses $b_i = A_i$. *Adaptive* coding sets $b_i = A_i - \hat{e}(\mathbf{x}_i)$, incorporating the estimated propensity score directly into the treatment multiplier; this is the parameterisation adopted in the **bcf** package. Finally, the *invariant* (parameter-expanded) coding of Hahn et al. (2020, Section 5.2) assigns group-specific intercepts $b_0, b_1 \sim \mathcal{N}(0, 1/2)$ and writes $y_i = \mu(\mathbf{x}_i) + b_{A_i}\,\tilde{\tau}(\mathbf{x}_i) + \varepsilon_i$, where b_0 and b_1 are updated within the Gibbs sampler via conjugate normal updates and the treatment effect is recovered as $\tau(\mathbf{x}_i) = (b_1 - b_0)\,\tilde{\tau}(\mathbf{x}_i)$. This parameterisation is invariant to the labelling of treatment and control. All four options are available in **ShrinkageTrees** via the `treatment_coding` argument.

For survival outcomes, we adopt an accelerated failure time (AFT) formulation. Let T_i denote the event time, C_i the censoring time, $Y_i = \min(T_i, C_i)$ the observed follow-up time, and $\delta_i \in \{0, 1\}$ the event indicator. The prediction model becomes

$$\log T_i = \sum_{j=1}^{m} g(\mathbf{x}_i;\, T_j, H_j) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2).$$

Right-censored observations ($\delta_i = 0$) are handled by imputing $\log T_i$ from a truncated normal distribution within the Gibbs sampler. **ShrinkageTrees** also supports interval-censored outcomes, where the event time is known only to lie in an interval $(l_i, r_i]$; these are imputed analogously. The causal model extends directly to the AFT setting by replacing y_i with $\log T_i$.

Posterior computation

All models are fitted using Markov chain Monte Carlo (MCMC). The outer loop is a Gibbs sampler that cycles through updates of the tree topologies, the leaf step heights, the error variance σ^2, and — where applicable — the shrinkage parameters and censored event times. Trees are updated one at a time via Bayesian backfitting (Chipman et al., 2010): at each sweep, a partial residual is formed by subtracting the contributions of all other trees from the response, and tree j is then updated conditional on the current state of all other trees. The tree topology is modified through Metropolis–Hastings proposals that either grow a terminal node into an internal node with two new leaves, or prune an internal node back to a leaf. Each proposal is accepted or rejected according to the standard Metropolis–Hastings ratio, which balances the likelihood improvement against the prior on tree structure.

Under the standard BART and Dirichlet (DART) priors, where the leaf parameters have a conjugate normal prior, the marginal likelihood of a proposed tree can be computed in closed form and the step heights are integrated out analytically. This yields the birth–death sampler described in Chipman et al. (2010); a detailed algorithmic exposition is given in Kapelner and Bleich (2016). When global–local shrinkage priors such as the horseshoe are placed on the step heights, this conjugacy is lost: the local and global scale parameters λ_ℓ and τ must be sampled explicitly, and the tree proposals become reversible jump MCMC moves that account for the change in the number of scale parameters when a leaf is added or removed. The details of this reversible jump construction, including the proposal distributions and acceptance ratios, are given in Jacobs et al. (2025).

3 Related Packages

Several R packages implement Bayesian tree ensembles. We review five CRAN packages and compare their capabilities to **ShrinkageTrees**. Table 1 provides a summary.

BART (Sparapani et al., 2021) is the most comprehensive general-purpose BART package. It supports continuous, binary, and multinomial outcomes, and is, at the time of writing, the only other CRAN package to offer Bayesian tree ensembles for survival data. Survival models include a discrete-time hazard formulation (`surv.bart()`), an AFT formulation (`abart()`), competing risks (`crisk.bart()`), and recurrent events (`recur.bart()`). The package also supports Dirichlet splitting priors via `sparse = TRUE`. However, **BART** does not provide global–local shrinkage priors on the leaf step heights, causal forest models, or interval-censored survival outcomes. Multi-chain MCMC is not natively supported; the `mc.*` functions parallelise a single chain rather than running independent chains.

dbarts (Dorie, 2023) provides a fast, lightweight BART sampler for continuous and binary outcomes. Its key feature is an updatable sampler object: predictors and responses can be modified

mid-sampling, which makes **dbarts** well suited as a conditional model within a larger Gibbs sampler. The package supports multi-chain MCMC but does not implement survival models, causal forests, or alternative leaf priors.

bcf (Hahn et al., 2020) implements Bayesian Causal Forests with separate prognostic and treatment effect forests for continuous outcomes. It places half-Cauchy and half-Normal hyperpriors on the leaf scales of the two forests, respectively. The package does not support survival endpoints, horseshoe priors, or Dirichlet splitting.

stochtree is a recent package that implements BART, XBART, BCF, and XBCF for continuous, binary, and ordinal outcomes. It supports variance forests for heteroskedastic modelling and provides model serialisation for warm-starting. However, **stochtree** does not implement AFT survival models, Dirichlet splitting priors, or horseshoe shrinkage on the leaf step heights.

SoftBart (Linero and Yang, 2018) replaces the hard axis-aligned splits of standard BART with smooth logistic splitting rules, yielding a posterior mean that is a continuous function of the covariates. This can improve performance when the true regression surface is smooth, but introduces additional computational cost and a fundamentally different regularisation mechanism. **ShrinkageTrees** exclusively uses hard decision splits, as in classical BART; extending the horseshoe and Dirichlet priors to soft splitting rules is an interesting direction for future work but is beyond the scope of this package.

At the time of writing, **ShrinkageTrees** is the only implementation that combines global–local shrinkage priors on the step heights with Bayesian tree ensembles for survival and causal inference. It is also the only package to support interval-censored survival outcomes within a Bayesian tree ensemble framework and the only package to offer causal forests for survival endpoints.

Table 1: Comparison of Bayesian tree ensemble packages in R.

Package	Survival	Interval Censoring	Causal (BCF)	Dirichlet (DART)
BART	Yes	No	No	Yes
dbarts	No	No	No	No
bcf	No	No	Yes	No
stochtree	No	No	Yes	No
SoftBart	No	No	No	No
ShrinkageTrees	Yes	Yes	Yes	Yes

4 Package Architecture

ShrinkageTrees is available on CRAN and on GitHub at `https://github.com/tijn-jacobs/ShrinkageTrees`. The package is implemented in R with a C++ backend via **Rcpp** (Eddelbuettel and François, 2011). In this section we describe the design and implementation.

Fitting functions and S3 classes

The package provides four primary fitting functions and four convenience wrappers for common model configurations. Table 2 lists all eight functions. The primary functions accept arguments that control the outcome type (`outcome_type`), prior specification (`prior_type`, `local_hp`, `global_hp`), tree structure (`number_of_trees`, `power`, `base`), and MCMC settings (`N_post`, `N_burn`, `n_chains`).

`ShrinkageTrees()` is the most general prediction model function. It accepts a `prior_type` argument that selects the leaf prior: `"horseshoe"` (per-tree global scale), `"horseshoe_fw"` (forest-wide global scale), `"half-cauchy"` (local shrinkage only), `"standard"` (conjugate normal, as in classical BART), or `"dirichlet"` (Dirichlet splitting, as in DART). `HorseTrees()` is a convenience wrapper that fixes the prior to the horseshoe and exposes a single tuning parameter k controlling the half-Cauchy scales via $\alpha_\tau = \alpha_\lambda = k/\sqrt{m}$. The causal functions `CausalShrinkageForest()` and `CausalHorseForest()` mirror this design for the causal BCF formulation, with independent prior configurations for the prognostic and treatment effect forests.

All prediction model functions return an object of class `"ShrinkageTrees"`; all causal model functions return `"CausalShrinkageForest"`. Both classes provide `print()`, `summary()`, `predict()`, and `plot()` methods. The `summary()` method reports posterior summaries for σ, variable inclusion probabilities, and — for causal models — the ATE with a credible interval. The `predict()` method re-runs the MCMC sampler on the new covariate matrix using the training data and hyperparameters

Table 2: Fitting functions in **ShrinkageTrees**. The first four are primary functions; the last four are convenience wrappers.

Function	Description
`ShrinkageTrees()`	Prediction model with user-specified leaf prior (horseshoe, half-Cauchy, standard, Dirichlet)
`HorseTrees()`	Prediction model with horseshoe prior; single tuning parameter k
`CausalShrinkageForest()`	BCF causal model with user-specified leaf priors for μ and τ forests
`CausalHorseForest()`	BCF causal model with horseshoe prior on both forests
`SurvivalBART()`	AFT survival with conjugate normal leaf prior (standard BART)
`SurvivalDART()`	AFT survival with Dirichlet splitting prior (DART)
`SurvivalBCF()`	AFT survival with BCF decomposition and conjugate normal leaf prior
`SurvivalShrinkageBCF()`	AFT survival with BCF decomposition and horseshoe leaf prior

stored in the fitted object; it always generates posterior samples internally, regardless of whether `store_posterior_sample` was set during fitting. The returned `ShrinkageTreesPrediction` object contains posterior means and credible intervals. The `plot()` method produces MCMC diagnostics (traceplots and posterior densities for σ), variable importance plots, and — for causal models — ATE and CATE visualisations. Plotting uses **ggplot2**, which is a suggested but not required dependency.

Rcpp backend

We implement the computationally intensive components of the Gibbs sampler in C++ via **Rcpp**. This includes the Metropolis-within-Gibbs updates for tree topology (grow, prune, and change moves), the conditional updates for the leaf step heights, the horseshoe augmentation steps for the local and global scale parameters, the Gibbs update for σ, and the data augmentation for censored observations. The R layer handles input validation, data preprocessing (centering, scaling, log-transforming survival times), hyperparameter calibration, S3 class construction, and post-processing (back-transformation of predictions).

Internally, the C++ code dispatches between two forest implementations depending on the prior. Priors that require reversible jump proposals (`"horseshoe"`, `"horseshoe_fw"`, `"half-cauchy"`) use a `Forest` class with paired grow–prune moves that satisfy detailed balance. The `"standard"` and `"dirichlet"` priors use a separate `StanForest` class with birth–death moves, which is simpler and faster when global scale updates are not needed.

The scale mixture prior on the step heights is encapsulated in a C++ class `ScaleMixture` that follows the strategy pattern. An abstract base class `EtaPrior` defines a common interface — proposing new local parameters, computing the log prior and proposal densities needed for the Metropolis–Hastings ratio, and optionally performing a global update — while concrete subclasses implement the prior-specific sampling logic. The current subclasses are `FixedVariance` (conjugate normal, used by BART and DART), `HalfCauchy` (local shrinkage only), `Horseshoe` (per-tree global–local shrinkage), and `Horseshoe_fw` (forest-wide global shrinkage). A factory function selects the appropriate subclass at run time based on the user's `prior_type` argument. Because the tree update and reversible jump routines interact with the prior only through the `ScaleMixture` interface, adding a new global–local shrinkage prior — for example a regularised horseshoe or a Dirichlet–Laplace prior — requires only implementing a new `EtaPrior` subclass and registering it in the factory, without modifying the sampler logic.

Multi-chain MCMC

All primary fitting functions accept an `n_chains` argument. When `n_chains > 1`, independent chains are dispatched in parallel via `parallel::mclapply()` using `min(n_chains, parallel::detectCores())` cores. After sampling, the posterior draws from all chains are pooled: sample matrices are concate-

nated row-wise and posterior means are recomputed from the pooled draws. The returned object is a standard `ShrinkageTrees` or `CausalShrinkageForest` instance, so all downstream methods work without modification. Per-chain diagnostics (acceptance ratios, σ traces) are retained for convergence assessment.

Convergence diagnostics via coda

Assessing MCMC convergence is an essential part of any Bayesian workflow, yet most BART implementations leave this task entirely to the user. **ShrinkageTrees** provides a bridge to the **coda** ecosystem (Plummer et al., 2006) through an `as.mcmc.list()` S3 method. This method extracts the posterior draws of σ — split by chain — and wraps each chain in a `coda::mcmc` object, returning a `coda::mcmc.list`. Because **coda** is a suggested rather than required dependency, all functionality remains available when it is not installed. When it is available, `summary()` automatically reports the effective sample size (ESS) and — for multi-chain fits — the Gelman–Rubin $\hat{R}$ statistic. Users who require more detailed diagnostics can pass the `mcmc.list` object directly to any **coda** function, including `geweke.diag()`, `heidel.diag()`, `autocorr.plot()`, and `gelman.plot()`, obtaining a complete convergence assessment without custom post-processing code.

5 Application: Ovarian Cancer Survival

We demonstrate the package on the `ovarian` dataset shipped with **ShrinkageTrees**. This dataset is curated from The Cancer Genome Atlas ovarian cancer cohort [TCGA-OV; Cancer Genome Atlas Research Network (2011)] and contains overall survival times, a binary treatment indicator, clinical covariates, and expression levels of 2,000 genes selected by median absolute deviation. Ovarian cancer is one of the most lethal gynaecological malignancies, accounting for approximately 85% of ovarian cancer deaths in its high-grade serous form (Cancer Genome Atlas Research Network, 2011). The five-year survival rate is below 50%. First-line treatment involves platinum-based chemotherapy, and gene expression patterns are known to be associated with survival duration and chemotherapy response (Cancer Genome Atlas Research Network, 2011). The choice between carboplatin and cisplatin — two platinum agents with different toxicity profiles — remains a central clinical question. The dataset provides a challenging high-dimensional causal survival problem, with gene expression data yielding $p \gg n$, that exercises the main features of the package. We first fit prediction models for survival, then extend the analysis to causal inference and fit a model to estimate the heterogenuous treaetment effect.

Data and setup

```
library(ShrinkageTrees)
data("ovarian")

clin      <- ovarian$clinical
time      <- clin$OS_time
status    <- clin$OS_event
treatment <- clin$treatment

X <- cbind(age        = clin$age,
           figo_stage = clin$figo_stage,
           tumor_grade = clin$tumor_grade,
           ovarian$X)

cat("n =", nrow(clin), "| p =", ncol(X),
    "| event rate =", round(mean(status), 2),
    "| carboplatin =", sum(treatment), "/ cisplatin =", sum(1 - treatment), "\n")
```

The dataset comprises 357 patients with 2,000 gene expression features and three clinical covariates (age, FIGO stage, tumour grade). We hold out 20% of the patients for evaluating predictive accuracy.

```
set.seed(2025)
train_idx <- sample(seq_len(nrow(clin)), size = floor(0.8 * nrow(clin)))
test_idx  <- setdiff(seq_len(nrow(clin)), train_idx)

X_train <- X[train_idx, ];  X_test <- X[test_idx, ]
```

```
time_train <- time[train_idx];  time_test <- time[test_idx]
status_train <- status[train_idx];  status_test <- status[test_idx]
```

Survival prediction models

We fit two prediction models on the training data: a standard BART model via `SurvivalBART()` and a Horseshoe Forest via `HorseTrees()`. Both use an AFT formulation with right-censoring and two independent MCMC chains. The BART model serves as a baseline; the Horseshoe Forest applies global–local shrinkage to the leaf step heights, which is expected to improve regularisation in this high-dimensional setting where the number of predictors far exceeds the sample size.

```
fit_bart <- SurvivalBART(
  time           = time_train,
  status         = status_train,
  X_train        = X_train,
  X_test         = X_test,
  number_of_trees = 50,
  N_post         = 5000,
  N_burn         = 5000,
  n_chains       = 2,
  store_posterior_sample = TRUE,
  verbose        = FALSE
)

fit_horse <- HorseTrees(
  y              = time_train,
  status         = status_train,
  X_train        = X_train,
  X_test         = X_test,
  outcome_type   = "right-censored",
  timescale      = "time",
  number_of_trees = 50,
  k              = 0.1,
  N_post         = 5000,
  N_burn         = 5000,
  n_chains       = 2,
  store_posterior_sample = TRUE,
  verbose        = FALSE
)
```

The `print()` and `summary()` methods provide a concise overview of the fitted model, including the outcome type, MCMC settings, acceptance ratio, and posterior summaries for σ.

```
print(fit_horse)
summary(fit_horse)
```

We evaluate predictive accuracy on the held-out test set using the concordance index [C-index; Harrell et al. (1996)], where a value of 1 indicates perfect discrimination and 0.5 corresponds to random prediction.

```
library(survival)
c_bart  <- concordance(Surv(time_test, status_test) ~ fit_bart$test_predictions)
c_horse <- concordance(Surv(time_test, status_test) ~ fit_horse$test_predictions)

cat("Test C-index — SurvivalBART:", round(c_bart$concordance, 3),
    " HorseTrees:", round(c_horse$concordance, 3), "\n")
```

Both models achieve a test C-index of approximately 0.60, indicating modest but above-chance discrimination — a reasonable result given the extremely high-dimensional setting ($p = 2{,}003 \gg n = 357$). The Horseshoe Forest and standard BART perform comparably here, consistent with the horseshoe prior providing regularisation benefits primarily in estimation and uncertainty quantification rather than in point prediction summaries.

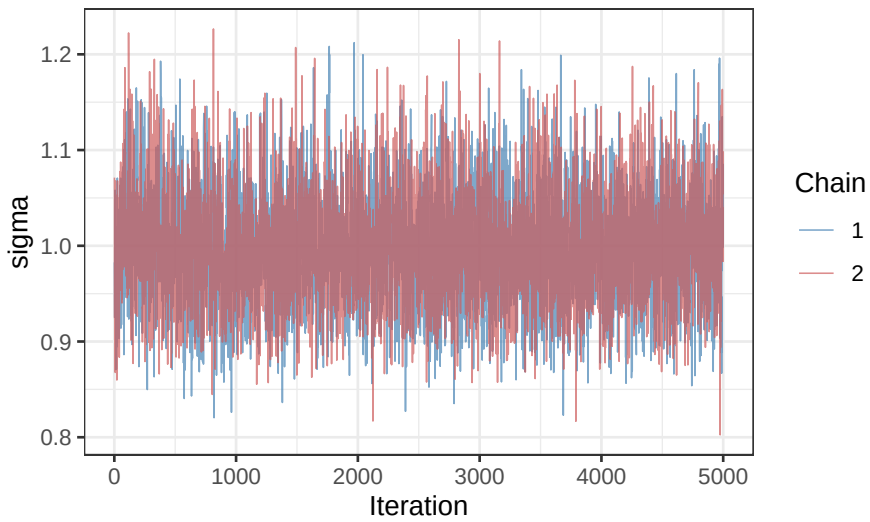

Figure 2: Traceplot of the posterior draws of σ for the Horseshoe Forest model. The two chains are shown in different colours.

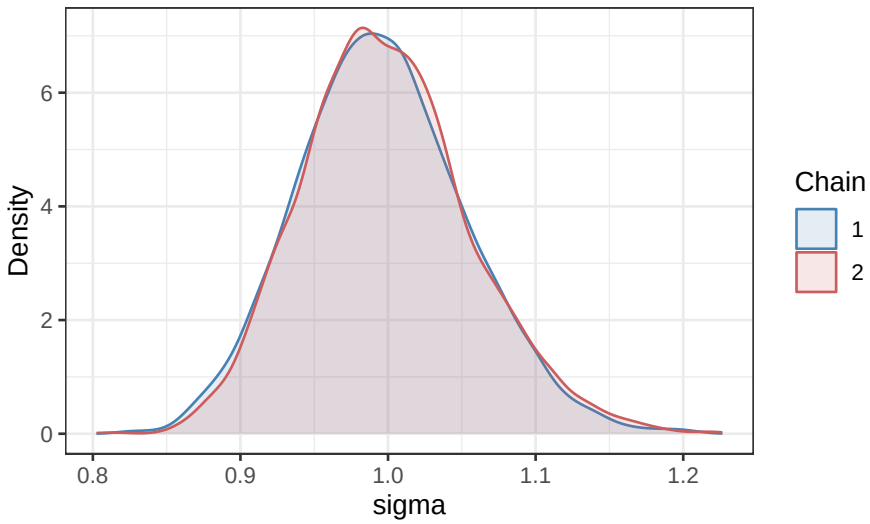

Figure 3: Posterior density of σ for the Horseshoe Forest model, estimated separately for each chain.

Convergence diagnostics

We assess convergence using σ traceplots and posterior densities. With `n_chains = 2`, each chain is plotted separately, allowing visual assessment of mixing and chain agreement. The `summary()` method automatically reports the effective sample size and Gelman–Rubin $\hat{R}$ statistic when the **coda** package is installed.

```
plot(fit_horse, type = "trace")
plot(fit_horse, type = "density")
```

Figures 2 and 3 show good mixing: the two chains overlap throughout the sampling period and yield nearly identical posterior densities for σ. The `summary()` output confirms convergence with $\hat{R} = 1$ and an effective sample size of 3,754.

Posterior survival curves

We identify two test-set patients at opposite ends of the predicted risk spectrum: the patient with the highest predicted survival time (best prognosis) and the patient with the lowest (worst prognosis).

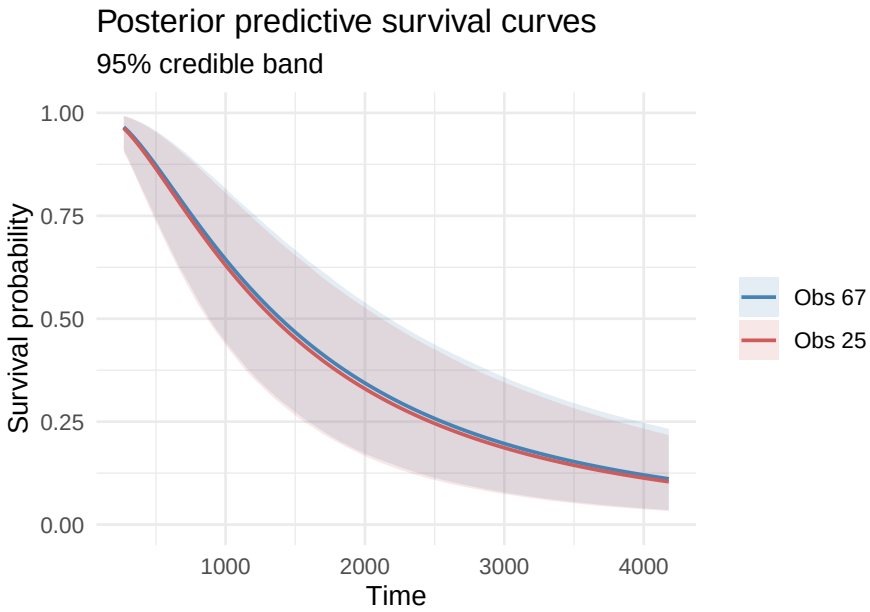

Figure 4: Posterior survival curves for two test-set patients at opposite ends of the predicted risk spectrum, with pointwise 95% credible bands.

```
pred <- predict(fit_horse, newdata = X_test)

idx_best  <- which.max(pred$mean)
idx_worst <- which.min(pred$mean)

cat("Best prognosis  - age:", X_test[idx_best, "age"],
    " stage:", X_test[idx_best, "figo_stage"],
    " grade:", X_test[idx_best, "tumor_grade"], "\n")
cat("Worst prognosis - age:", X_test[idx_worst, "age"],
    " stage:", X_test[idx_worst, "figo_stage"],
    " grade:", X_test[idx_worst, "tumor_grade"], "\n")
```

Individual posterior survival curves under the AFT log-normal model $S(t \mid \mathbf{x}) = 1 - \Phi((\log t - \hat{\mu}(\mathbf{x}))/\hat{\sigma})$ are plotted with pointwise 95% credible bands.

```
plot(pred, type = "survival", obs = c(idx_best, idx_worst))
```

Figure 4 shows the posterior survival curves for the two selected patients. The separation between the curves reflects the model's ability to distinguish risk profiles despite the heavy shrinkage imposed in this high-dimensional setting. We also plot the population-averaged posterior survival curve across all training observations, overlaid with the Kaplan–Meier estimate for visual calibration.

```
plot(fit_horse, type = "survival", km = TRUE)
```

In Figure 5, the model-based survival curve closely tracks the Kaplan–Meier estimate during the observed follow-up period, with expected divergence in the right tail where data are sparse.

Causal survival analysis

We now estimate heterogeneous treatment effects of carboplatin versus cisplatin using `CausalHorseForest()`, which fits a BCF model with horseshoe shrinkage on both the prognostic and treatment effect forests. We first estimate propensity scores from a logistic regression on the clinical covariates. Using the full gene expression matrix for propensity estimation would risk overfitting given $p \gg n$; clinical covariates alone capture the main sources of confounding by indication.

```
ps_model   <- glm(treatment ~ age + factor(figo_stage) + factor(tumor_grade),
                  family = binomial, data = clin)
```

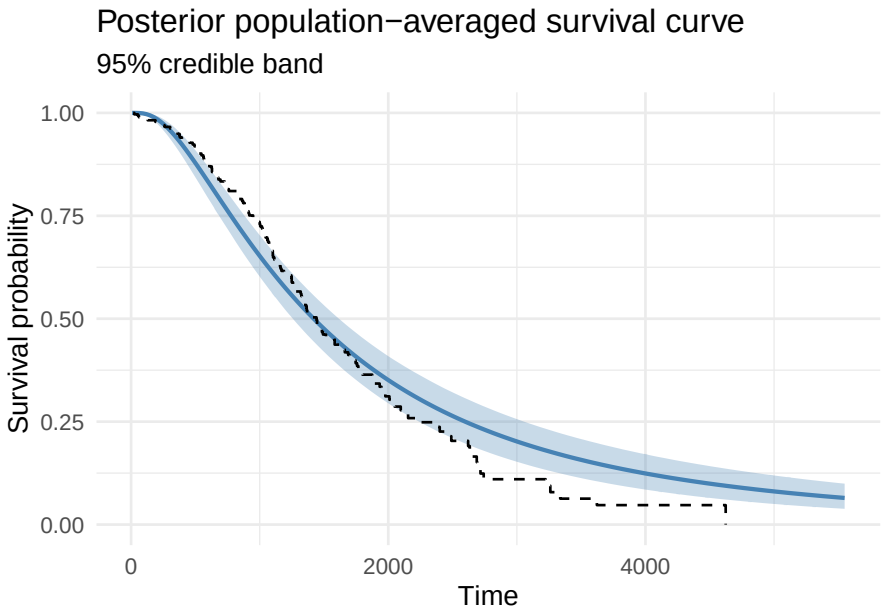

Figure 5: Population-averaged posterior survival curve (solid line) with 95% credible band (shaded) and Kaplan–Meier estimate (dashed).

```
propensity <- predict(ps_model, type = "response")
X_control  <- cbind(propensity = propensity, X)

set.seed(2025)
fit_causal <- CausalHorseForest(
  y                        = log(time),
  status                   = status,
  X_train_control          = X_control,
  X_train_treat            = X,
  treatment_indicator_train = treatment,
  outcome_type             = "right-censored",
  timescale                = "log",
  number_of_trees          = 50,
  k                        = 0.1,
  N_post                   = 5000,
  N_burn                   = 5000,
  n_chains                 = 2,
  store_posterior_sample   = TRUE,
  verbose                  = FALSE
)
summary(fit_causal)
```

The `summary()` method reports the posterior average treatment effect (ATE) on the log-survival scale with a 95% credible interval. Here we pass `log(time)` directly with `timescale = "log"`, so all treatment effect estimates remain on the log-survival scale. A positive ATE indicates that carboplatin is associated with longer survival times.

Treatment effect posteriors

The `plot()` method provides visualisations of the treatment effect posterior. The ATE density plot shows the full posterior distribution of the average treatment effect, while the CATE caterpillar plot displays patient-level treatment effect estimates sorted by posterior mean with 95% credible intervals.

```
plot(fit_causal, type = "ate")
plot(fit_causal, type = "cate")
```

Figure 6 shows the treatment effect posteriors. The ATE posterior is centred near zero on the log-survival scale, and the individual CATEs cluster tightly around the ATE with narrow credible

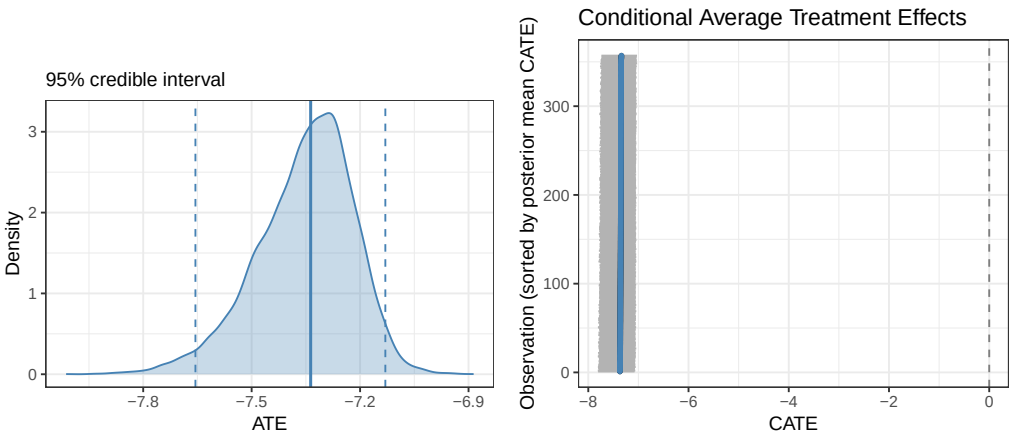

Figure 6: Left: posterior density of the average treatment effect (ATE) of carboplatin versus cisplatin on the log-survival scale, with 95% credible interval (dashed lines). Right: patient-level conditional average treatment effects (CATEs) sorted by posterior mean with 95% credible intervals; the dashed line marks zero (no effect).

intervals. This suggests a relatively homogeneous treatment effect across the patient population — consistent with both platinum agents acting through similar mechanisms.

Interval-censored survival

Although the ovarian dataset contains right-censored observations, **ShrinkageTrees** also supports interval-censored outcomes via `outcome_type = "interval-censored"` with `left_time` and `right_time` arguments following the convention of `survival::Surv(type = "interval2")`. See `?ShrinkageTrees` for details and examples.

6 Discussion and Conclusion

We have introduced **ShrinkageTrees**, an R package for Bayesian tree ensembles that combines global–local shrinkage priors with survival analysis and causal inference. The package decouples structural regularisation from parametric shrinkage, allowing practitioners to tailor the regularisation strategy to the problem at hand. To our knowledge, it is the first CRAN package to offer Bayesian causal forests for survival endpoints, horseshoe priors on tree ensemble step heights, and interval-censored survival outcomes within a single framework.

The package has several limitations that suggest directions for future work. Currently, all covariates must be numeric: continuous, binary, and ordered categorical variables (encoded as integers) are handled naturally by the axis-aligned splitting rules, but unordered categorical variables must be dummy-encoded by the user. This can be suboptimal for high-cardinality factors, where native subset splits — as implemented in **bartMachine** (Kapelner and Bleich, 2016) — may be more efficient. The current AFT formulation assumes normally distributed errors; extending the framework to alternative error distributions (e.g. logistic, log-logistic) would broaden its applicability. The Gibbs sampler can be slow to mix in very high-dimensional settings; incorporating warm-starting or accelerated sampling strategies could improve computational efficiency.

ShrinkageTrees is available on CRAN and actively maintained on GitHub at `https://github.com/tijn-jacobs/ShrinkageTrees`. The methodological foundations are developed in Jacobs et al. (2025).

7 Funding

Tijn Jacobs and Stéphanie van der Pas received funding from the European Research Council (ERC) under the European Union's Horizon Europe program under Grant agreement No. 101074082. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council Executive Agency. Neither the European Union nor the granting authority can be held responsible for them.

Bibliography

Cancer Genome Atlas Research Network. Integrated genomic analyses of ovarian carcinoma. *Nature*, 474:609–615, 2011. doi: 10.1038/nature10166. [p7]

A. Caron, G. Baio, and I. Manolopoulou. Estimating individual treatment effects using non-parametric regression models: A review. *Journal of the Royal Statistical Society Series A: Statistics in Society*, 185(3): 1115–1149, 2022. doi: 10.1111/rssa.12889. [p3]

C. M. Carvalho, N. G. Polson, and J. G. Scott. The horseshoe estimator for sparse signals. *Biometrika*, 97(2):465–480, 2010. doi: 10.1093/biomet/asq017. [p1, 3]

H. A. Chipman, E. I. George, and R. E. McCulloch. Bayesian CART model search. *Journal of the American Statistical Association*, 93(443):935–948, 1998. doi: 10.1080/01621459.1998.10473750. [p2]

H. A. Chipman, E. I. George, and R. E. McCulloch. BART: Bayesian additive regression trees. *The Annals of Applied Statistics*, 4(1):266–298, 2010. doi: 10.1214/09-AOAS285. [p1, 2, 3, 4]

V. Dorie. *dbarts: Discrete Bayesian Additive Regression Trees Sampler*, 2023. URL https://CRAN.R-project.org/package=dbarts. R package version 0.9-23. [p4]

V. Dorie, J. Hill, U. Shalit, M. Scott, and D. Cervone. Automated versus do-it-yourself methods for causal inference: Lessons learned from a data analysis competition. *Statistical Science*, 34(1):43–68, 2019. doi: 10.1214/18-STS667. [p1]

D. Eddelbuettel and R. François. Rcpp: Seamless R and C++ integration. *Journal of Statistical Software*, 40(8):1–18, 2011. doi: 10.18637/jss.v040.i08. [p1, 2, 5]

P. R. Hahn, J. S. Murray, and C. M. Carvalho. Bayesian regression tree models for causal inference: Regularization, confounding, and heterogeneous effects. *Bayesian Analysis*, 15(3):965–1056, 2020. doi: 10.1214/19-BA1195. [p1, 3, 4, 5]

F. E. Harrell, K. L. Lee, and D. B. Mark. Multivariable prognostic models: Issues in developing models, evaluating assumptions and adequacy, and measuring and reducing errors. *Statistics in Medicine*, 15 (4):361–387, 1996. doi: 10.1002/(SICI)1097-0258(19960229)15:4<361::AID-SIM168>3.0.CO;2-4. [p8]

J. L. Hill. Bayesian nonparametric modeling for causal inference. *Journal of Computational and Graphical Statistics*, 20(1):217–240, 2011. doi: 10.1198/jcgs.2010.08162. [p1]

T. Jacobs, W. N. van Wieringen, and S. L. van der Pas. Horseshoe forests for high-dimensional causal survival analysis. *arXiv preprint arXiv:2507.22004*, 2025. doi: 10.48550/arXiv.2507.22004. [p1, 3, 4, 12]

A. Kapelner and J. Bleich. bartMachine: Machine learning with Bayesian additive regression trees. *Journal of Statistical Software*, 70(4):1–40, 2016. doi: 10.18637/jss.v070.i04. [p4, 12]

A. R. Linero. Bayesian regression trees for high-dimensional prediction and variable selection. *Journal of the American Statistical Association*, 113(522):626–636, 2018. doi: 10.1080/01621459.2016.1264957. [p1, 3]

A. R. Linero and Y. Yang. Bayesian regression tree ensembles that adapt to smoothness and sparsity. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 80(5):1087–1110, 2018. doi: 10.1111/rssb.12293. [p5]

M. Plummer, N. Best, K. Cowles, and K. Vines. CODA: Convergence diagnosis and output analysis for MCMC. *R News*, 6(1):7–11, 2006. URL https://journal.r-project.org/archive/. [p7]

N. G. Polson and J. G. Scott. On the half-Cauchy prior for a global scale parameter. *Bayesian Analysis*, 7 (4):887–902, 2012. doi: 10.1214/12-BA730. [p3]

V. Ročková and E. Saha. On the theory of BART. *arXiv preprint arXiv:1910.00597*, 2020. [p2]

R. Sparapani, C. Spanbauer, and R. McCulloch. Nonparametric machine learning and efficient computation with Bayesian additive regression trees: The BART R package. *Journal of Statistical Software*, 97(1):1–66, 2021. doi: 10.18637/jss.v097.i01. [p1, 4]

Tijn Jacobs
Vrije Universiteit Amsterdam
Department of Mathematics
Amsterdam, The Netherlands
https://tijn-jacobs.github.io/
ORCiD: 0009-0003-6188-9296
t.jacobs@vu.nl

Stéphanie van der Pas
Vrije Universiteit Amsterdam
Department of Mathematics
Amsterdam, The Netherlands
https://www.stephanievanderpas.nl/
ORCiD: 0000-0002-2448-5378
s.l.vander.pas@vu.nl

Wessel van Wieringen
Vrije Universiteit AmsterdamAmsterdam University Medical Centre
Department of Mathematics, Amsterdam, The Netherlands
Epidemiology and Data Science, Amsterdam, The Netherlands
https://www.math.vu.nl/~wvanwie/
ORCiD: 0000-0002-5100-9123
w.n.vanwieringen@vu.nl